=== UNIT 5: POINTERS, DYNAMIC MEMORY ALLOCATION & STRING MANIPULATION ===

>> 5.1 Pointer Concepts, Pointer Arithmetic and Indirection

Pointers are variables that store the physical memory addresses of other variables.
* Pointer Declaration and Dereferencing: int *ptr = &var;. The address-of operator (&) extracts memory address; dereference (indirection) operator (*) accesses or mutates value stored at that address.
* Pointer Arithmetic: Scaled by sizeof(data_type). If ptr holds address 1000 and points to a 4-byte integer: ptr + 1 evaluates to 1004. Subtraction (ptr2 - ptr1) yields number of elements between pointers.
* Pointers and Arrays: Array name decays to constant pointer to first element: *(arr + i) is mathematically identical to arr[i].
* Pointers to Pointers (Double Pointers): int **pptr = &ptr;. Used in dynamic 2D array allocation and modifying caller pointer variables.

>> 5.2 Dynamic Memory Allocation (DMA) in Heap Space

Standard variables reside in Stack memory (fixed size, automatic lifetime). Dynamic memory is allocated at runtime in Heap memory via stdlib.h routines:
* malloc(size_t size): Allocates single uninitialized contiguous block of 'size' bytes. Memory contains garbage values. Returns void* on success, NULL on failure: int *p = (int*)malloc(n * sizeof(int));.
* calloc(size_t num, size_t size): Allocates contiguous memory for 'num' elements of 'size' bytes each, and automatically initializes every bit to ZERO. Prevents garbage bugs.
* realloc(void *ptr, size_t new_size): Resizes previously allocated memory block. If expanding, may extend current block or allocate new larger block elsewhere, copy existing contents, free old block, and return new pointer.
* free(void *ptr): Releases allocated heap block back to operating system memory pool. Dangling Pointer Bug: A pointer still pointing to memory that has been freed. Fix: Assign ptr = NULL immediately after free(ptr).
* Memory Leak: Occurs when heap memory is allocated but never freed, and all pointer references are lost. Over time, exhausts system RAM.

>> 5.3 String Manipulation Without Library Functions

In C, a string is a one-dimensional character array terminated by a null character ('\0', ASCII 0).
* Standard Functions (string.h): strlen(), strcpy(), strcat(), strcmp(), strstr().
* Implementing string algorithms from first principles reinforces pointer traversal and null-termination handling.

[PROGRAM 1: Implementing strlen, strcpy, strcat, strcmp from Scratch]

Problem: Write custom implementations of the four core C string library functions using pure pointer arithmetic without including string.h.

Source Code:

```
#include <stdio.h>
size_t my_strlen(const char *s) {
const char *p = s;
while (*p != '\0') p++;
return p - s;
}
char* my_strcpy(char *dest, const char *src) {
char *ret = dest;
while ((*dest++ = *src++) != '\0');
return ret;
}
char* my_strcat(char *dest, const char *src) {
char *ret = dest;
while (*dest != '\0') dest++;
while ((*dest++ = *src++) != '\0');
return ret;
}
int my_strcmp(const char *s1, const char *s2) {
while (*s1 && (*s1 == *s2)) { s1++; s2++; }
return *(const unsigned char*)s1 - *(const unsigned char*)s2;
}
int main() {
char str1[50] = "Hello ";
char str2[] = "World!";
printf("Length of str1: %zu\n", my_strlen(str1));
my_strcat(str1, str2);
printf("Concatenated: %s (Length: %zu)\n", str1, my_strlen(str1));
printf("Compare 'Apple' vs 'Banana': %d\n", my_strcmp("Apple", "Banana"));
return 0;
}
```

Output: Length of str1: 6 | Concatenated: Hello World! (Length: 12) | Compare 'Apple' vs 'Banana': -1.

[PROGRAM 2: Dynamic 2D Array Allocation Using Pointer-to-Pointer (int)]**

Problem: Write an ANSI C program to dynamically allocate an (R x C) matrix in heap

memory using int**, initialize it with multiplication tables, and properly deallocate every block to prevent memory leaks.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
int rows = 3, cols = 4;
int **matrix = (int**)malloc(rows * sizeof(int*));
if (matrix == NULL) { printf("Row allocation failed!\n"); return 1; }
for (int i = 0; i < rows; i++) {
matrix[i] = (int*)malloc(cols * sizeof(int));
if (matrix[i] == NULL) { printf("Column allocation failed!\n"); return 1; }
}
for (int i = 0; i < rows; i++) {
for (int j = 0; j < cols; j++) {
matrix[i][j] = (i + 1) * (j + 1);
printf("%4d", matrix[i][j]);
}
printf("\n");
}
for (int i = 0; i < rows; i++) {
free(matrix[i]);
matrix[i] = NULL;
}
free(matrix);
matrix = NULL;
printf("Dynamic matrix memory deallocated successfully!\n");
return 0;
}
```

Output:

```
1 2 3 4
2 4 6 8
3 6 9 12
Dynamic matrix memory deallocated successfully!
```

[PROGRAM 3: Dynamic Array Reallocation (realloc) Demonstration]

Problem: Write a program that reads integers dynamically until the user inputs -1, using realloc() to double array capacity when full.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
int capacity = 2, count = 0;
```

```
int *arr = (int*)malloc(capacity * sizeof(int));
int sampleInputs[] = { 10, 20, 30, 40, 50, -1 };
for (int i = 0; sampleInputs[i] != -1; i++) {
if (count == capacity) {
capacity *= 2;
int *temp = (int*)realloc(arr, capacity * sizeof(int));
if (temp == NULL) { free(arr); return 1; }
arr = temp;
printf("Resized buffer capacity to %d integers\n", capacity);
}
arr[count++] = sampleInputs[i];
}
printf("Stored %d elements: ", count);
for (int i = 0; i < count; i++) printf("%d ", arr[i]);
printf("\n");
free(arr);
return 0;
}
```

Output: Resized buffer capacity to 4 integers | Resized buffer capacity to 8 integers | Stored 5 elements: 10 20 30 40 50.

[PROGRAM 4: Reversing Words in a String In-Place]

Problem: Write an in-place string reversal function that reverses the words in a sentence without auxiliary string buffers.

Source Code:

```
#include <stdio.h>
void reverseSubstr(char *begin, char *end) {
char temp;
while (begin < end) {
temp = *begin; *begin++ = *end; *end-- = temp;
}
}
void reverseWords(char *s) {
char *word_begin = s;
char *temp = s;
while (*temp) {
temp++;
if (*temp == '\0') {
reverseSubstr(word_begin, temp - 1);
} else if (*temp == ' ') {
reverseSubstr(word_begin, temp - 1);
word_begin = temp + 1;
}
```

```
}
reverseSubstr(s, temp - 1);
}
int main() {
char text[] = "Biotechnology Engineering LPU";
printf("Original: %s\n", text);
reverseWords(text);
printf("Reversed Words: %s\n", text);
return 0;
}
```

Output: Original: Biotechnology Engineering LPU | Reversed Words: LPU Engineering Biotechnology.

>> 5.4 University Review & Examination Questions

* Q1: Contrast malloc(), calloc(), realloc(), and free() in C regarding their arguments, initialization, and error handling.
Answer: (1) malloc(size_t size): Takes single argument specifying total bytes. Allocates contiguous memory on Heap without initialization (contents are garbage). Returns void* pointer to first byte, or NULL on failure. (2) calloc(size_t num, size_t size): Takes two arguments: number of elements and size of each. Allocates memory and clears all bits to ZERO. Useful for arrays and numeric buffers to prevent garbage bugs. Returns NULL on failure. (3) realloc(void *ptr, size_t new_size): Resizes an existing heap block pointed to by ptr. May expand in-place or relocate the block to a new heap location, preserving existing data up to minimum of old and new size. If ptr is NULL, behaves like malloc(new_size). Returns pointer to reallocated memory or NULL on failure. (4) free(void *ptr): Deallocates previously allocated heap memory, returning it to runtime heap pool. Does not zero out pointer, creating dangling pointer if not explicitly set to NULL.
* Q2: Explain the concepts of Dangling Pointers and Memory Leaks with code examples and mitigation techniques.
Answer: A Dangling Pointer points to memory that has already been deallocated (freed) or gone out of scope. Example: 'int *p = (int*)malloc(sizeof(int)); free(p); *p = 50; // UNDEFINED BEHAVIOR'. Mitigation: Always assign 'p = NULL;' immediately following 'free(p);' and verify 'if (p != NULL)' before dereferencing. A Memory Leak occurs when dynamically allocated heap memory is no longer referenced by any pointer but was never released with free(). Example: 'void leak() { int *p = (int*)malloc(100); } // Pointer p destroyed on exit, but 100 bytes remain stranded on Heap'. Over time, leaks accumulate and cause program termination due to Out-Of-Memory errors. Mitigation: Always pair every malloc/calloc call with a corresponding free() invocation.

>> 5.5 Key Takeaways and Revision Points

* Pointer arithmetic increments by sizeof(data_type); array name decays to pointer to

element 0.
* malloc allocates uninitialized bytes; calloc zeros memory; realloc resizes heap blocks.
* Dangling pointers point to freed memory; prevent by setting pointers to NULL post-free.
* C strings are null-terminated character arrays; string functions operate by advancing pointers until '\0'.